

Lecture 07: Date and Functions in JavaScript

The JavaScript Date Object:

Method 1: The Date Object

JavaScript manages dates and times using the built-in Date object. When initialized as a new object, it captures the exact current moment.

```
const now = new Date();  
console.log(now.toString());
```

Under the hood, JavaScript tracks time in UTC. However, executing methods like `.toString()` automatically converts and formats the output into the device's Local Timezone.

How does the engine know the time?

The JavaScript engine does not track time internally. Instead, it queries the device's **Operating System (OS)**.

How does the device get the time?

Computers and mobile devices automatically synchronize their internal clocks with global internet servers using **NTP (Network Time Protocol)**.

Extracting Date & Time Components:

JavaScript provides specific getter methods to retrieve exact parts of a date.

Method	Description
<code>.getFullYear()</code>	Returns the full 4-digit year.
<code>.getMonth()</code>	Returns the month (0 - 11).
<code>.getDate()</code>	Returns the day of the month (1 - 31).
<code>.getDay()</code>	Returns the day of the week (0 - 6).
<code>.getHours()</code>	Returns the hour (0 - 23).
<code>.getMinutes()</code>	Returns the minutes (0 - 59).
<code>.getSeconds()</code>	Returns the seconds (0 - 59).

The Zero-Index Legacy Inconsistency:

Months and days of the week are zero-indexed, but days of the month start at 1. This inconsistency is a legacy design flaw inherited from Java's original Date API, which JavaScript could never fix afterward without breaking backward compatibility with existing websites.

Important Note: Due to these quirks, modern development environments strictly replace native Date object manipulation with robust libraries like `date-fns` or `Day.js`.

Method 2: Timestamps

Instead of creating a full Date object, a lightweight numeric timestamp can be retrieved using the static `.now()` method.

```
const timestamp = Date.now();  
console.log(timestamp);
```

- **The UNIX Epoch:** The timestamp represents the total number of milliseconds elapsed since January 1, 1970 (UTC).
- **Negative Timestamps:** Timestamps can be negative. A negative value simply represents a date occurring before January 1, 1970.

Converting Timestamp to Date:

```
const date = new Date(timestamp);
console.log(date);
```

Which timestamp should a server store?

Systems must strictly rely on server time (typically stored in UTC) because users can easily manipulate the clock on their local devices.

Can a JavaScript timestamp overflow?

JavaScript stores numbers as 64-bit double-precision floats. The maximum safe integer in JavaScript allows timestamps to accurately represent dates up to the year **285,616**. Therefore, a timestamp overflow in JavaScript is practically impossible.

Custom Dates:

Specific dates can be constructed manually by passing parameters in a strict order. (year, month, date, hours, minutes, seconds, milliseconds)

```
const myDate = new Date(2026, 6, 1, 0, 0, 0, 0);
console.log(myDate.toString());
```

Functions in JavaScript:

What is a function?

A function is a reusable block of code designed to perform a specific task.

Why is it needed?

It adheres to the DRY (Don't Repeat Yourself) principle. Logic is written once, and rather than duplicating the code, the function is executed repeatedly as needed.

Different Methods to Write Functions:

Method 1: Standard Function Declarations (Named Functions)

```
// Basic Function
function greeting(){
  console.log("Hello World!");
}
greeting();

// Function with Parameters and Arguments
function addTwoNumbers(num1, num2){
  return num1 + num2;
}
console.log(addTwoNumbers(2,3));

// Function utilizing the Rest Operator for infinite arguments
function addManyNumbers(...arr){
  let sum = 0;
  for(const num of arr) {
    sum += num;
  }
  return sum;
}
console.log(addManyNumbers(1, 2, 3, 4, 5));
```

Method 2: Function Expressions (Anonymous Functions)

```
const addNumber = function(num1,num2){  
    return num1+num2;  
}
```

Method 3: Arrow Functions (Modern ES6)

```
// Standard Arrow Function  
const addNumber = (num1, num2) => {  
    return num1 + num2;  
}
```

```
// Variation 1: Implicit Return (No curly braces or 'return' keyword needed)  
const add = (num1, num2) => num1 + num2;
```

```
// Variation 2: Single Parameter (Parentheses can be omitted)  
const square = num => num * num;
```

Method 4: IIFE (Immediately Invoked Function Expression)

A function that runs instantly the moment it is defined. It is wrapped in parentheses to treat it as an expression, followed by () to execute it.

```
(function hello(){  
    console.log("Hello");  
})();
```

Callback Functions:

A Callback is simply a function passed as an argument directly *inside* another function.

Advantage: Prevents hardcoding and promotes highly modular, reusable code.

Real-World Callback Example:

```
function zomatoRestaurant(){  
    console.log("Restaurant is preparing the order...");  
}  
  
function blinkitWarehouse(){  
    console.log("Packing the order for user...");  
}  
  
function payment(amount, callback){  
    console.log(`Processing payment of ₹${amount}...`);  
    console.log("Payment is done!");  
    callback();  
}  
  
payment(500,zomatoRestaurant);  
payment(1000,blinkitWarehouse);
```